

## **Stage 5. Instruction Set Architecture**

- ❖ **Address Calculation Unit for Relative and Index Addressing Modes**

- ❖ **Stack and Stack Pointer (SP)**

Stack Related Commands

## Address Calculation Unit for Relative and Index Addressing Modes

---

It was mentioned that the effective address is calculated by adding the one-byte offset value following instructions with relative and index addressing modes with the program counter (PC) or index register (IX).

We can use the arithmetic logic unit (ALU) we have to perform this addition. However, before this collection process, we will need to throw the data in the accumulator and data register into the stack so that we do not lose it, and after calculating the effective address, we will need to restore this information from the stack again.

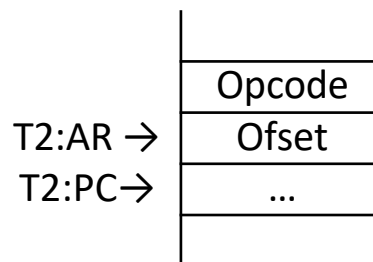
These operations will extend the execution time of instructions with these addressing modes. Performing the effective address calculation in a separate unit instead of the ALU will prevent this loss of time. By giving  $1010_2$  to the inputs of the decoders used in road selection, the effective address information calculated by this unit is transferred to the bus.

## Address Calculation Unit for Relative and Index Addressing Modes

---

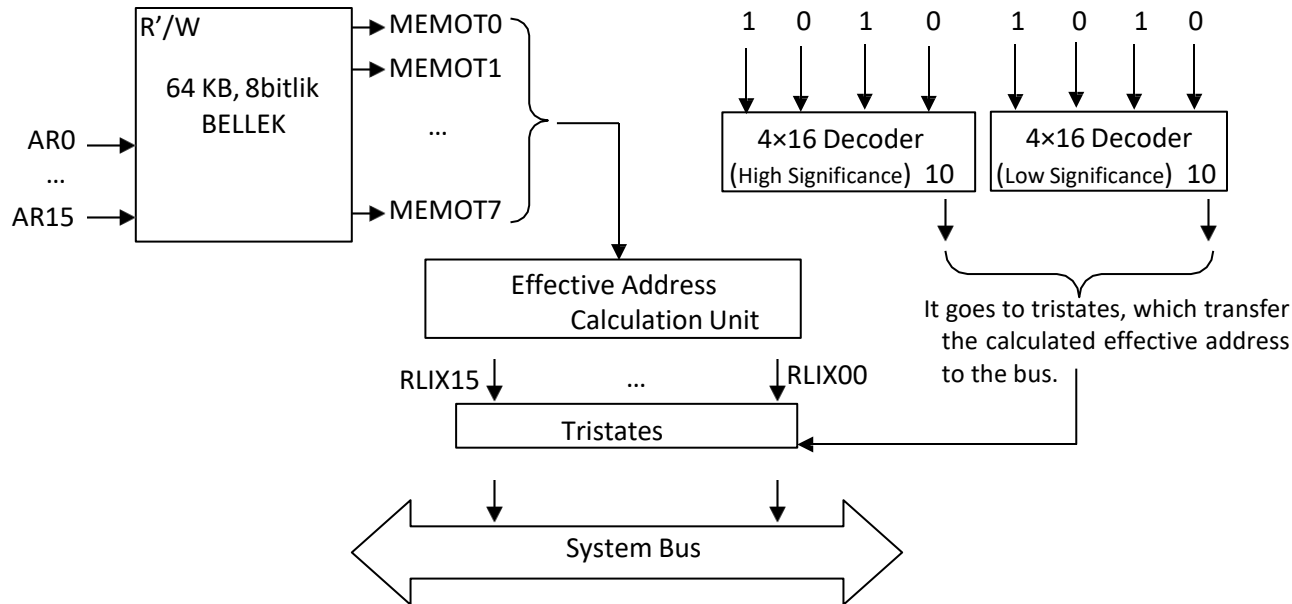
PC and IX are 16-bit registers, while memory is 8-bit. In order to calculate the effective address, these registers must be summed with the offset part of the instruction. The offset part of instructions using index addressing mode is considered as an unsigned number, and the offset part of instructions with relative mode is considered as a signed number (in 2's complement form).

While commands with these addressing modes are being executed, the offset value at the output of the memory will be transmitted to the active address calculation unit, since AR points to the offset value in memory in the T2 time period and the memory is directly connected to AR.



# Address Calculation Unit for Relative and Index Addressing Modes

The 1 byte offset value at the output of the memory is named MEMOT0 ... MEMOT7.



In fact, even if there are no instructions with relative and index addressing modes, information always goes to the active address calculation unit. For example, in the T2 time slot of an instruction with direct addressing mode, the high-meaning part of the address information of the operand is transmitted to this unit. Although an effective address is always calculated, effective address information is transferred to the path only in index and relative addressing modes.

## Calculation of Active Address

---

Since 16-bit addition operations will be performed in the active address calculation unit, the 8-bit offset value must be expressed as 16 bits.

In index addressing mode, since the offset value is an unsigned number, 8 0s are added to expand the offset to 16 bits.

In relative addressing mode, since the offset value is a signed number, the offset value is expanded depending on the sign bit. In other words, expansion is made with the most significant part (MEMOT7) of the offset value read from memory. As a result of these operations, no change occurs in the actual value of the offset value.

# Calculation of Active Address

**Example:** Let the offset value be 85h. Let's calculate the effective address for both addressing modes.

According to relative addressing mode, this number is a negative number. Value;  $85h = 1000\ 0101_2 \rightarrow$  If we take the 2's complement and put the '-' sign in front of it,

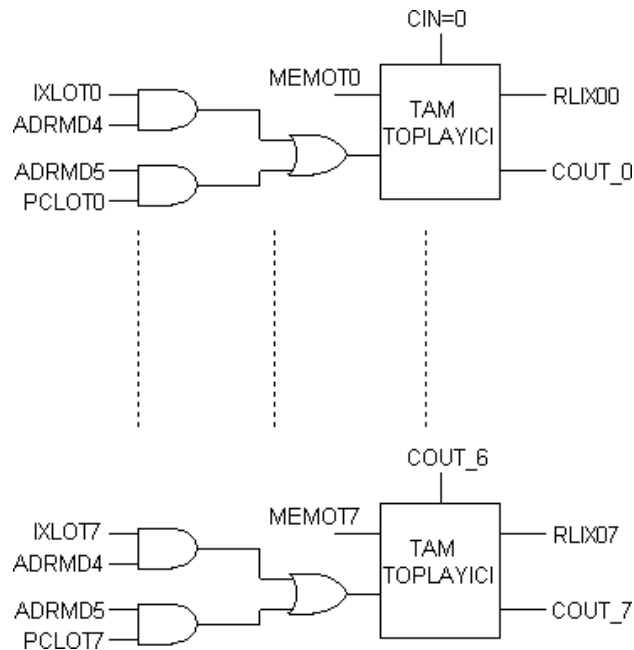
-  $(0111\ 1011_2) = -123$  'tür.

Let's assume the PC is 1234h. In this case, the effective address is calculated as follows; Since the sign bit of the offset value is 1, 8 1's are placed to the left of the offset value. In this case, the 16-bit version of the offset value is FF85h. The decimal equivalent of this corresponds to -123.

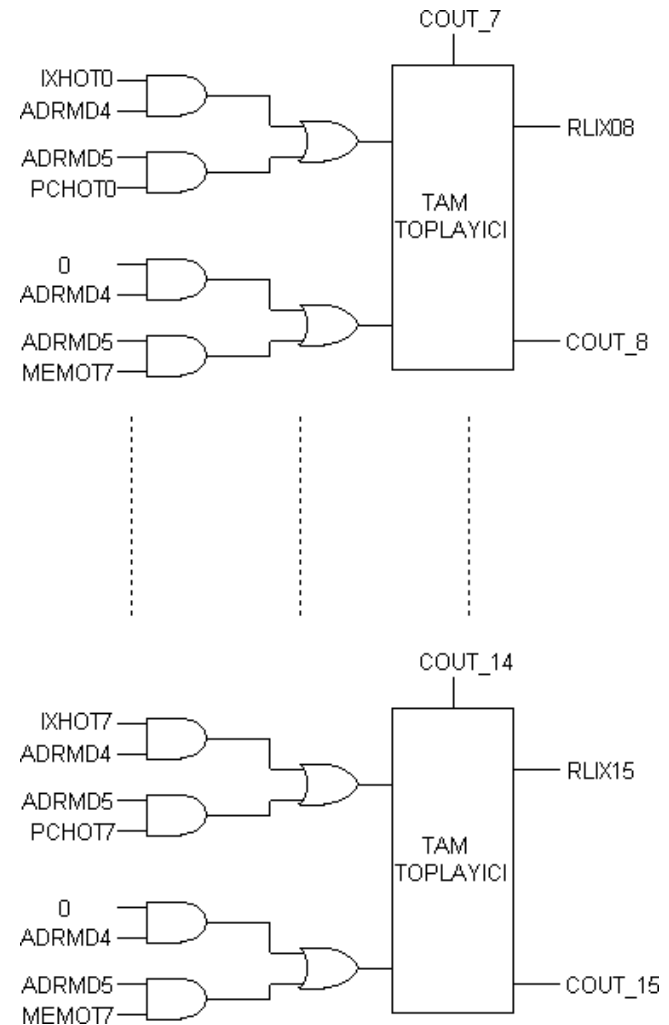
Effective address =  $1234h + FF85h = 11B9h$ .

According to the index addressing mode, this number is considered unsigned and corresponds to the decimal value 133. Before calculating the effective address, 8 0s are placed to the left of the offset value. In this case, the 16-bit version of the offset value is 0085h. If we take the value of IX as 1234h, Effective address =  $1234h + 0085h = 12B9h$ .

# Hardware Structure of the Active Address Calculation Unit



**Low significance part**



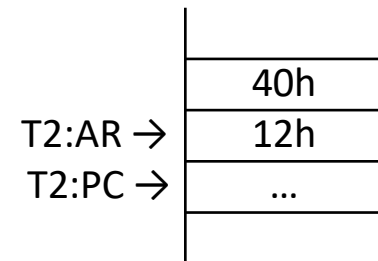
**High significance part**

## Example: Steps of the ADD command with index mode

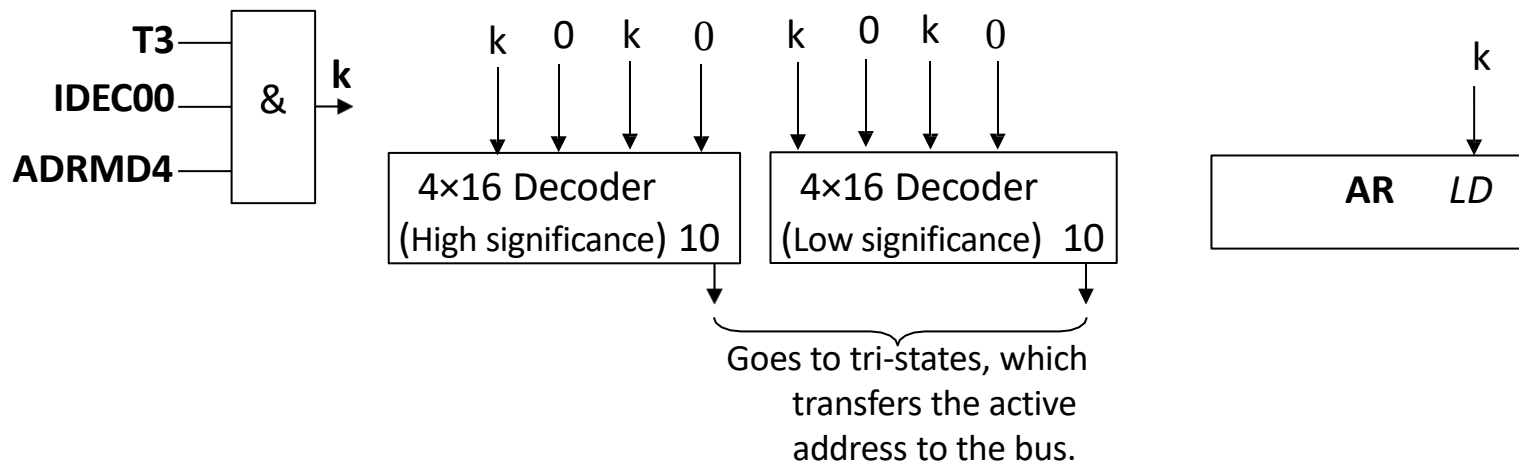
Like the ADD \*12h command. The symbol '\*' is used to represent index addressing. It is a 2 byte command. 12h in the command is the offset value. 12h will be added to the contents of the index register and the effective address will be calculated.

The opcode of the ADD instruction for index mode is  $0\ 100\ 0000_2 = 40h$ .

| MICRO PROCESSING STEPS OF THE COMMAND |                                                              |
|---------------------------------------|--------------------------------------------------------------|
| T3. IDEC00.ADRMD4:                    | $AR \leftarrow \text{Active Address}$                        |
| T4. IDEC00.ADRMD4:                    | $DR_H \leftarrow M[AR], AR \leftarrow AR+1$                  |
| T5. IDEC00.ADRMD4:                    | $DR_L \leftarrow M[AR]$                                      |
| T6. IDEC00.ADRMD4:                    | $AC \leftarrow AC+DR, C \leftarrow C_{out}, SC \leftarrow 0$ |



### Hardware implementation of the microprocessing step performed in the T3 timing slice



## Stack and Stack Pointer (SP)

---

When there is any branching, the value of PC should be thrown onto the stack to be used on the return from the subroutine. When an interrupt occurs, the contents of the PC, IX, AC and CCR registers should be saved to the stack, and after the interrupt program is executed, these values should be taken from the stack and loaded back into the relevant registers.

For this process, first a stack region is determined in memory. SP indicates the address where the data will be placed. As data is added to the stack, the value of SP is reduced by 1. Similarly, as data is retrieved from the stack, it is incremented by 1. So, the stack type used in our basic computer system is LIFO (Last-In First-Out).

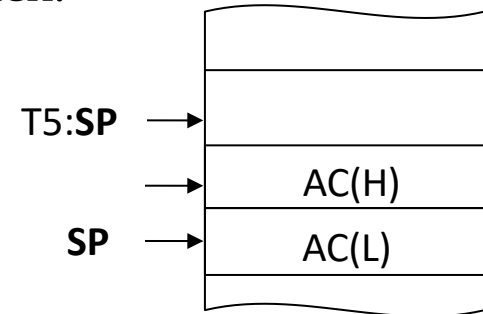
Compilers of high-level programs can use the stack to hold local and global variables. Our basic computer system has PSH (put on stack) and PUL (get from stack) commands to perform these tasks. It is also used in mathematical operations to ensure transaction priority.

In summary, the stack offers the possibility of branching into a particularly large number of subroutines.

## Stack Related Commands

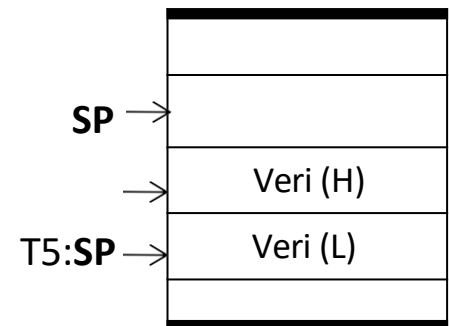
**PSH (Push):** It saves the data in the accumulator to the location pointed by the stack pointer. It has a inherite addressing mode. First the low and then the high significance part of the acumulator is placed on the stack.

| MICRO PROCESSING STEPS OF THE COMMAND |                                                                 |
|---------------------------------------|-----------------------------------------------------------------|
| <b>T3. IDEC06.ADRMD0:</b>             | $AR \leftarrow SP$                                              |
| <b>T4. IDEC06.ADRMD0:</b>             | $M[AR] \leftarrow AC_L, SP \leftarrow SP-1, AR \leftarrow AR-1$ |
| <b>T5. IDEC06.ADRMD0:</b>             | $M[AR] \leftarrow AC_H, SP \leftarrow SP-1, SC \leftarrow 0$    |



**PUL (Pop):** It copies the data where the stack pointer points to the accumulator. It has a inherite addressing mode. Since the value in the stack cannot be loaded directly to the AC, it is first loaded to DR and then to the AC.

| MICRO PROCESSING STEPS OF THE COMMAND |                                                                 |
|---------------------------------------|-----------------------------------------------------------------|
| <b>T3.IDEC07.ADRMD0:</b>              | $SP \leftarrow SP+1$                                            |
| <b>T4. IDEC07.ADRMD0:</b>             | $AR \leftarrow SP$                                              |
| <b>T5. IDEC07.ADRMD0:</b>             | $DR_H \leftarrow M[AR], SP \leftarrow SP+1, AR \leftarrow AR+1$ |
| <b>T6. IDEC07.ADRMD0:</b>             | $DR_L \leftarrow M[AR]$                                         |
| <b>T7.IDEC07.ADRMD0:</b>              | $AC \leftarrow DR, SC \leftarrow 0$                             |



## Stack Related Commands

**BSR (Branch to SubRoutine):** It provides a short branch to the subprogram at the active address calculated unconditionally. It uses relative addressing mode. It requires the contents of the PC to be dumped onto the stack.

| MICRO PROCESSING STEPS OF THE COMMAND |                                                                       |
|---------------------------------------|-----------------------------------------------------------------------|
| T3. IDEC15.ADRMD5:                    | $TR \leftarrow AR$ (address of offset saved)                          |
| T4. IDEC15.ADRMD5:                    | $AR \leftarrow SP$                                                    |
| T5. IDEC15.ADRMD5:                    | $M[AR] \leftarrow PC_L$ , $AR \leftarrow AR-1$ , $SP \leftarrow SP-1$ |
| T6. IDEC15.ADRMD5:                    | $M[AR] \leftarrow PC_H$ , $SP \leftarrow SP-1$                        |
| T7. IDEC15.ADRMD5:                    | $AR \leftarrow TR$ (address of offset restored)                       |
| T8. IDEC15.ADRMD5:                    | $PC \leftarrow$ Active address, $SC \leftarrow 0$                     |

The reason for the microtransactions in steps T3 and T7; The effective address calculation unit needs the offset value. Since AR is directly connected to memory, the offset value will be ready at the output of the memory along with the T7 cycle and this value will go to the active address calculation unit.

## Stack Related Commands

**JSR (Jump to SubRoutine):** Unconditionally allows long branching to the subroutine. It uses direct and index addressing modes.

Microprocessing steps for direct addressing mode;

| MICRO PROCESSING STEPS OF THE COMMAND |                                                                    |
|---------------------------------------|--------------------------------------------------------------------|
| <b>T3. IDEC24.ADRMD2:</b>             | $TR \leftarrow AR, PC \leftarrow PC+1$ (address information saved) |
| <b>T4. IDEC24.ADRMD2:</b>             | $AR \leftarrow SP$                                                 |
| <b>T5. IDEC24.ADRMD2:</b>             | $M[AR] \leftarrow PC_L, AR \leftarrow AR-1, SP \leftarrow SP-1$    |
| <b>T6. IDEC24.ADRMD2:</b>             | $M[AR] \leftarrow PC_H, SP \leftarrow SP-1$                        |
| <b>T7. IDEC24.ADRMD2:</b>             | $AR \leftarrow TR$ (beginning of address information restored)     |
| <b>T8. IDEC24.ADRMD2:</b>             | $TR_H \leftarrow M[AR], AR \leftarrow AR+1$                        |
| <b>T9. IDEC24.ADRMD2:</b>             | $TR_L \leftarrow M[AR]$                                            |
| <b>T10. IDEC24.ADRMD2:</b>            | $PC \leftarrow TR, SC \leftarrow 0$                                |

The microprocessing steps of JSR using index addressing mode and BSR instructions using relative addressing mode are the same. The difference is in the calculation of active addresses.

## Stack Related Commands

**RTS (ReTurn to Subroutine):** Returns from the subroutine to where the program flow left off. The RTS instruction is similar to the PUL instruction. The difference is that the information read from the stack is put into the PC instead of the AC.

| MICRO PROCESSING STEPS OF THE COMMAND |                                                                 |
|---------------------------------------|-----------------------------------------------------------------|
| T3. IDEC26.ADRMD0:                    | $SP \leftarrow SP+1$                                            |
| T4. IDEC26.ADRMD0:                    | $AR \leftarrow SP$                                              |
| T5. IDEC26.ADRMD0:                    | $PC_H \leftarrow M[AR], AR \leftarrow AR+1, SP \leftarrow SP+1$ |
| T6. IDEC26.ADRMD0:                    | $PC_L \leftarrow M[AR], SC \leftarrow 0$                        |